Adversarial Robustness for Deep Neural Networks

by

Muhammad Abdullah 011221256
Tasmia Sultana 0112230573
Rudro Kumar Bandhapadhay 011221118
Mezbah Uddin Emon 011221433
Mridul Kanti Paul 011221265
Md Shazzad Hossain Jiku 011221073

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
UNITED INTERNATIONAL UNIVERSITY

January 15, 2026

Contents

1 INTRODUCTION 3

1.1 PROBLEM STATEMENT 3
1.2 MOTIVATION AND SCOPE 4
1.3 OBJECTIVES 4

2 BACKGROUND 4

2.1 PRELIMINARIES 4
2.1.1 Adversarial Attacks 4
2.1.2 Projected Gradient Descent(PGD) 5
2.1.3 Standard supervised training 5
2.1.4 Defense Mechanisms 5
2.1.5 How It Spreads 6
2.1.6 How were they dealt 6
2.2 LITERATURE REVIEW 7
2.3 GAP ANALYSIS 8

3 PROJECT DESIGN 9

3.1 REQUIREMENT ENGINEERING 9
3.1.1 Functional Requirements 9
3.1.2 Non-Functional Requirements 9
3.1.3 Context Diagram 10
3.2 DESIGN 10
3.2.1 Proposed System Diagram 10
3.3 TASK ALLOCATION 11

CHAPTER 1

1 Introduction

In recent years, Deep Neural Networks have made a significant contribution to machine learning (ML). Like image classification, object detection, language translation, and many more. For big data and hardware acceleration, deep learning requires fewer hand-engineered features and expert knowledge. The number of real-world applications powered by DNN is increasing rapidly. Architecture for the interfaces between these stages is required. In particular, safety and security-focused applications are concerned about robustness to adversarial examples. Data points which have been adversarially perturbed to fool a model (Szegedy et al., 2013). So, our goal is to make the model accurate on the data and on perturbed versions of the data.

1.1 Problem Statement

DNNs work well with well-designed input samples, but a tiny modification to the input can cause a big problem. Deep learning is widely known as a "Black Box" technique; we know it performs well with limited, well-designed knowledge. To resolve this issue, many studies have been proposed to explain DNN. Adversarial examples for neural network models were discovered by (Szegedy et al.) and later (Goodfellow et al.) systematically introduced an approach to intrude on the robustness property of a neural network to make it misclassify a tampered input to a wrong result. Since then, neural networks have been shown to be vulnerable to adversarial attacks; consequently, research on neural network security has experienced explosive growth. Adversarial attacks entail purposeful alterations to provide input data that misleads deep learning models, resulting in inaccurate predictions.

1.2 Motivation and Scope

The primary goal is to ensure robustness against adversarial examples is essential for applications such as autonomous driving, healthcare diagnostics, and security systems.

1.3 Objectives

In this paper, authors will investigate and summarize approaches to generating adversarial examples, their applications in adversarial attacks, and corresponding countermeasures. We will explore the possible problem of adversarial examples. We will mainly focus on adversarial examples for image classification, object detection, and language translation.

CHAPTER 2

2 Background

Keeping all of these in mind, the main goal is to build a model that is robust to adversarial attacks using current or perturbed data. In this section, we will briefly introduce basic to advanced deep learning techniques and approaches related to adversarial examples.

2.1 Preliminaries

People in the DL community have been showing interest in both adversarial attack and Defenses. It caught attention because of its implications for autonomous vehicles, security systems, and the robustness of ML models. This section reviews key works in this domain, focusing on the foundational theories of adversarial attacks, prominent defense mechanisms, and their respective strengths and limitations.

2.1.1 Adversarial Attacks

First, we discuss some important works on adversarial attacks. The concepts of adversarial examples and the FGSM attack were introduced in (Goodfellow

et al. 2014). This paper proved that deep learning models can be fooled with a simple, single-step gradient-based attack, which initiated significant research in the field of adversarial machine learning and defense mechanisms. A major strength of this work is its clear demonstration that small, intentional perturbations to input data can lead deep learning models to confidently make inaccurate predictions. However, the scope of the proposed defense is limited, as adversarial training can be computationally expensive and may not generalize well to unseen attack types.

2.1.2 Projected Gradient Descent(PGD)

The Projected Gradient Descent (PGD) attack is essentially the same as the BIM (or IFGSM) attack. The only difference is that PGD initializes the example to a random point in the ball of interest (decided by the L norm) and does random restarts, while BIM initializes to the original point. However, while the defense is effective against first-order attacks like PGD, its performance against more sophisticated or adaptive attacks remains less explored.

2.1.3 Standard supervised training

Standard supervised training does not mandate that the chosen model will be resistant to adversarial examples. This property must be encoded into the training procedure somehow and that is where adversarial training comes in. It can be shown that by training on a mixture of adversarial and clean examples, a neural network could be regularized somewhat. In general, data augmentation uses techniques such as translation, rotation, etc in order to produce input samples similar to what might be expected in the test set.. However, the proposed defenses may be computationally intensive and might not scale well with larger datasets or more complex models.

2.1.4 Defense Mechanisms

The following are some well-known defense mechanisms against adversarial attacks proposed in the literature. The concept of defensive distillation, introduced in (Papernot et al. 2015), which proposes the technique builds upon the idea of knowledge distillation, where a smaller ”student” network is trained to replicate the behavior of a larger ”teacher” network. Perturbations, thereby improving its resistance to adversarial examples.

However, defensive distillation may not be effective against more sophisticated attacks that have emerged since its proposal.

However, a notable weakness is that the study primarily focuses on specific attack types and models, potentially limiting the generalizability of its findings to other multimodal attack strategies.

Looking at the papers that has elaborated on Deep Neural Network they concludes that : they're great at guessing things, but can we really trust them? People usually talk about two main problems: how easily they can be fooled and how hard it is to understand what they're doing.

2.1.5 How It Spreads

Back in 2013, Szegedy and his team first showed that you could mess with DNNs using adversarial examples. These are tiny changes to images that humans can't even see, but they make the computer guess wrong. Then, in 2017, Papernot and others showed that these tricks even work when you don't know anything about the model—it's like a trick you learn for one model can fool lots of others that were trained on similar stuff.

How They've Tried to Fight Back: The two main ways people have tried to fix this are called Adversarial Training and Defensive Distillation.Kurakin and others talked in 2016 about Adversarial Training, which is a muscle prposal. You just throw a bunch of these tricky examples into the training data.But even that can be beaten by just shaking things up a bit. Then there's Defensive Distillation, which Papernot's group came up with in 2016. It tries to make the model less sensitive by softening its output. But Carlini and Wagner showed in 2016 that it's not very strong against many types of attacks.

2.1.6 How were they dealt

To deal with the black box problem of DNN—meaning we don't know how they make decisions—some researchers have tried using simpler models to explain why a DNN makes a certain guess. Think of it like drawing a simple line to explain a complicated curve (Ribeiro et al., 2016). One natural way to do this is to look at how much the output changes when you tweak the input, which is called an input gradient (Baehrens et al., 2010). But these gradients are often too messy to understand. That's why people came up with ways to smooth them out, like SmoothGrad (Smilkov et al., 2017) and Integrated

Gradients (Sundararajan et al., 2016). These methods try to give a clearer picture by averaging out the changes over a small area.

A Brief Look at Gradient Regularization: The idea of keeping a network's output changes steady when the input changes—also known as double backpropagation—is actually pretty old, first talked about by Drucker and Le Cun in 1992. It was originally used to make models better at handling new data.

2.2 Literature Review

In the recent times, DNN has been widely used and the results are noteworthy in the dataset. But, the simplest and unnoticeable changes confuses the neural network in decision making. However, adversarial training has been proposed which will reduce the faults but is expensive and also limited. As a result, adversarial robustness is still a field of research where AI is aimed at a safe and reliable system.

This review goes over how neural network robustness is verified, as presented by Meng et al. (2022). The authors reviewed 39 papers, putting them into groups based on formal verification: property formalization, model reduction, and reasoning strategies.

1. Defining Properties The basic part of verification is setting up the robustness property, especially for adversarial attacks. Types of Perturbations: Research shows global perturbations (vectors covering the whole input) and regional perturbations (local spots or Trojan triggers). How They're Measured: The size of these perturbations is measured using norm functions, mostly the norm, which fits with the Fast Gradient Sign Method (FGSM) attack. What Robustness Means: Robustness is defined as a safety property where the model must always map an input and its changed versions to the same output within a set range.

2. Ways to Simplify Models A key idea in the literature is to simplifyneural networks into math models that existing verification algorithms can solve. The survey points out five main ways to do this: SAT/LP Encoding: Earlier work turned networks into Boolean satisfiability (SAT) or Linear Programming (LP) problems. This seemed good, but it caused a branch explosion because of the way ReLU activations work. MILP Encoding: To fix the branching problems, researchers started using Mixed-Integer Linear Programming (MILP). By giving integer variables to ReLU states, tools

like 'MIPVERIFY' kept things both correct and complete. QCQP/SDP Encoding: Some methods changed the problem into Quadratically Constrained Quadratic Programs (QCQP) or Semidefinite Programming (SDP). This allowed for looking at quadratic relationships but took a lot of computer power, often leading to complexity. Layer-wise Propagation Discretization: Methods like 'DLV' use discretization to send input values through each layer. While simpler, discretization can lose accuracy and doesn't promise completeness. Abstract Interpretation: A popular approach recently (DEEPPOLY) is to guess network behavior using abstract groups (boxes). This method focuses on being fast and scalable, but sometimes it loses accuracy (completeness).

2.3 Gap Analysis

Serial	**Paper**	**Gap**
1	Explaining and Harnessing Adversarial Examples (Goodfellow et al., 2015)	Only explains FGSM attacks; other adversarial methods are not discussed.
2	Towards Deep Learning Models Resistant to Adversarial Attacks (Madry et al., 2017)	Too much computational costs makes the approach difficult to scale to larger datasets.
3	Distillation as a Defense to Adversarial Perturbations (Papernot et al., 2016)	Instead of fundamentally improving model it reduces gradient visibility.
4	Adversarial Training for Free! (Shafahi et al., 2019)	Doesn't work better in unseen or adaptive attacks.
5	Fast is Better Than Free: Revisiting Adversarial Training (Andriushchenko et al., 2020)	Only works better in simple datasets like CIFAR-10.
6	Adversarial Robustness of Deep Neural Networks: A Survey (Meng et al., 2022)	Isn't suited for real world systems or large scale models.

CHAPTER 3

3 Project Design

In this chapter, we will discuss about the requirements of our own project, the methodology we used, and about the design of out project.

3.1 Requirement Engineering

Two types of requirements: -Functional Requirements -Non-Functional Requirements

3.1.1 Functional Requirements

A functional requirement defines a system or its component. It specifies "What should the software system do?". It is specified by User and a mandatory requirement. It helps to verify the functionality of the software. In this system, the users are researchers or students that train the deep learning models for adversarial attacks. Therefore, user requirements are defined in terms of model training and robustness evaluation.
1.Dataset Handling: The system should allow users to load a labeled image
2.Model Training: The system should allow users to train a DNN model using supervised learning algorithm.
3.Clean Data Evaluate: The system shall allow the user to evaluate the trained model's accuracy
4.Robustness Evaluation: The system shall allow the user to evaluate model performance on adversarial examples and measure robustness
5.Result Verfication: The system shall allow the user to compare clean accuracy results.

3.1.2 Non-Functional Requirements

A non-functional requirement defines the quality attribute of a software system. It places constraints on "How should the software system fulfill the functional requirements?".Non- functional requirement is specified by technical peoples e.g. Architect, Technical leaders and software developers and is not mandatory. It helps to verify the performance of the software. **Technical**

Requirements: 1.Performance:The system should complete model training and evaluation within a reasonable time on standard computing hardware.
2.Usability: The system should be easy to use for students and researchers with basic knowledge of deep learning.
3.Scalability: The system should support different dataset sizes and model configurations.
4.Reliability: The system should produce consistent results when the same inputs and parameters are used.
5.Maintainability: The system should be modular so that new attack or defense methods can be added easily.

3.1.3 Context Diagram

A context diagram is a high-level visual representation that illustrates the interactions between a system and its external entities. It defines the system's boundaries, showing what is inside and outside the system.

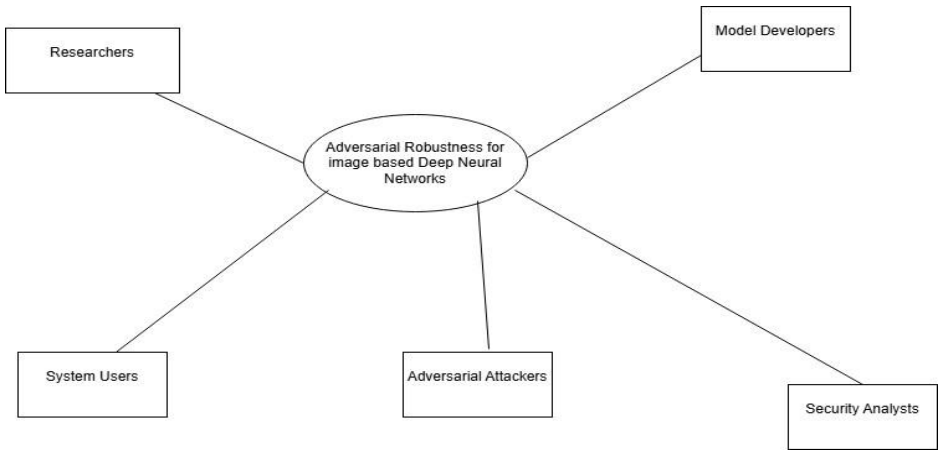

Figure 1: Context Diagram

3.2 Design

3.2.1 Proposed System Diagram

The proposed system illustrates a robust learning pipeline designed to improve model resilience against adversarial attacks. Clean training and test

data are first processed alongside adversarial examples generated by an attack generator. These inputs undergo feature extraction, after which the model is trained using robustness-oriented defense mechanisms. The trained robust model then performs predictions that are resistant to adversarial perturbations, ultimately producing correctly classified and robust output labels even in the presence of adversarial inputs.

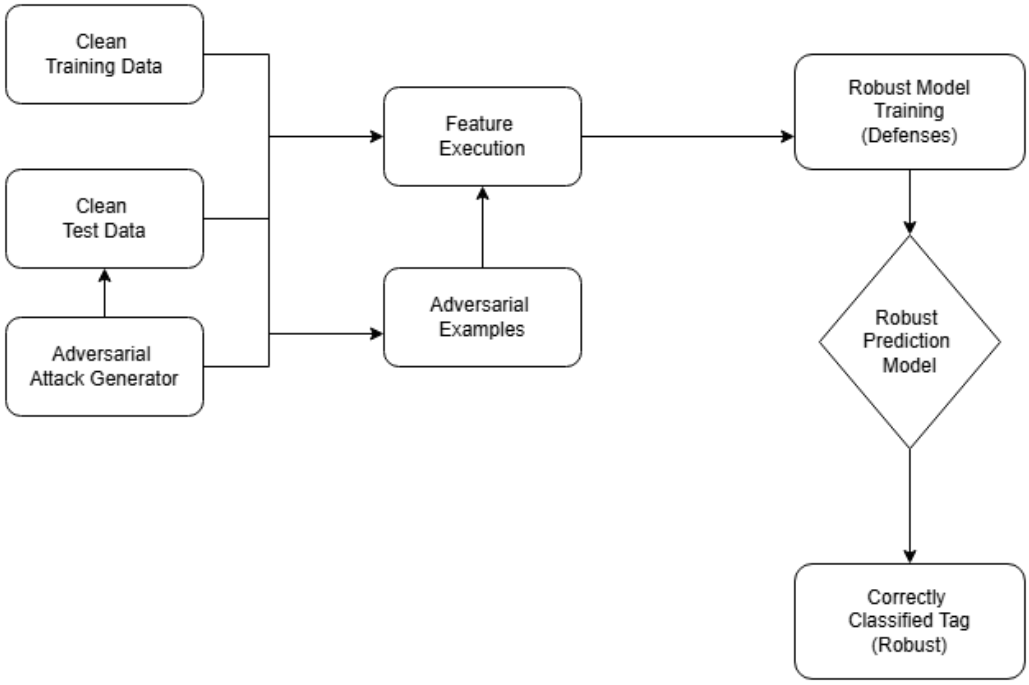

Figure 2: Proposed System Diagram

3.3 Task Allocation

Task allocation for this research project was managed using Jira to ensure structured coordination among the six group members. The project was divided into well-defined tasks and subtasks, which were assigned to individual members based on their roles and responsibilities. Jira enabled clear tracking of task progress, deadlines, and dependencies, facilitating transparency, accountability, and effective collaboration throughout the research workflow.

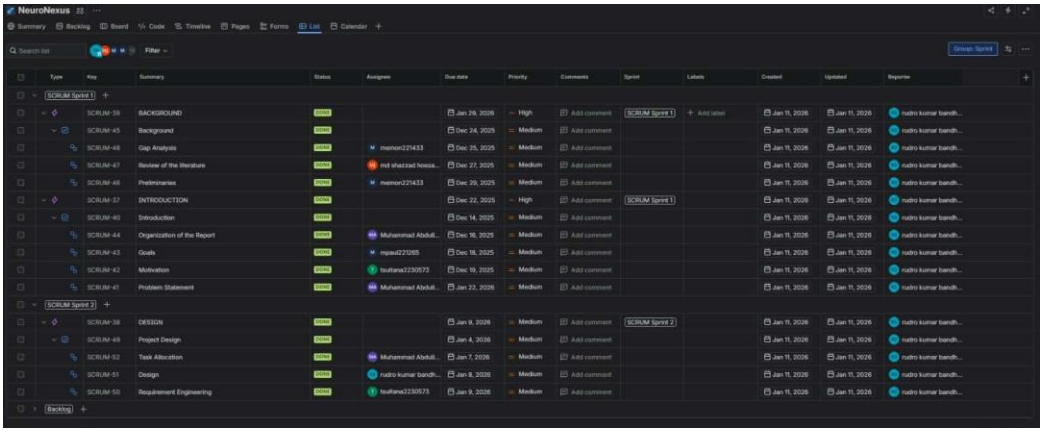

Figure 3: Task Allocation Graph